NGUYEN DEVELOPMENT · PHPDBADMIN

# Evolutionary Development — phpdbadmin

Timeboxes und Architekturentscheide ADR-001 bis ADR-011

| | |
|---|---|
| **Phase** | 04 Evolutionary Development |
| **Träger** | Nguyen Development (Einzelfirma), Hoang Nguyen · Lizenz MIT |
| **Vorgehen** | DSDM/AgilePM — Dokumentation in dokumentationen/, ein PDF je Phase |
| **Stand** | 29.10.2026, 00:47 · baut auf Commit 1c8f969 + Änderungen dieses Standes auf |
| **Quelle** | dokumentationen/ im Repo — dieses PDF wird bei jeder Änderung neu gebaut |

**Dieses Dossier enthält**


1. 04 — Evolutionary Development
2. Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen
3. Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber
4. Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber
5. Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf
6. Anhang E — ADR-005 — SQL-Editor: CodeMirror 6
7. Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel
8. Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund
9. Anhang H — ADR-008 — Anmeldung ohne Passwort
10. Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe
11. Anhang J — ADR-010 — Lizenz MIT
12. Anhang K — ADR-011 — PHP 8.5 als Mindestversion
13. Anhang L — Timebox 00 — Fundament
14. Anhang M — Timebox 01 — Katalog und Navigator
15. Anhang N — Timebox 02 — Datenraster und Filter

# Inhalt

**04 — Evolutionary Development**
Timeboxes
Architekturentscheide

**Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen**
Kontext
Entscheid
Optionen
Konsequenzen

**Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber**
Kontext
Entscheid
Optionen
Konsequenzen

**Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber**
Kontext
Entscheid
Optionen
Konsequenzen

**Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf**
Kontext
Entscheid
Optionen
Konsequenzen

**Anhang E — ADR-005 — SQL-Editor: CodeMirror 6**
Kontext
Entscheid
Optionen
Konsequenzen

**Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel**
Kontext
Entscheid
Optionen
Konsequenzen

**Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund**
Kontext
Entscheid
Optionen
Konsequenzen

**Anhang H — ADR-008 — Anmeldung ohne Passwort**
Kontext
Entscheid
Optionen
Konsequenzen
Nachtrag TB 00 (28.10.2026)

**Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe**
Kontext
Entscheid
Optionen
Konsequenzen

**Anhang J — ADR-010 — Lizenz MIT**
Kontext
Entscheid
Optionen
Konsequenzen

**Anhang K — ADR-011 — PHP 8.5 als Mindestversion**
Kontext
Entscheid
Optionen
Konsequenzen

**Anhang L — Timebox 00 — Fundament**

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Browser-Durchlauf
Entscheide in der Timebox
Commits
Offen
Gelernt

### Anhang M — Timebox 01 — Katalog und Navigator

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Tests
Commits
Gelernt

### Anhang N — Timebox 02 — Datenraster und Filter

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Tests
Commits
Gelernt

# 04 — Evolutionary Development

> **PDF der Phase:** evolutionary-development.pdf — bei jeder Änderung neu gebaut (`python3 dokumentationen/werkzeuge/pdf-bauen.py --nur 04`).

Timebox-Protokolle (`timebox-nn-titel.md`) und Architekturentscheide (`adr/ADR-nnn-titel.md`: Kontext → Optionen → Entscheid → Konsequenzen).

## Timeboxes

| TB | Inhalt | Anforderungen | Stand |
|---|---|---|---|
| **00 Fundament** | Starter-Kit bereinigt (kein Passwort, kein Dashboard, kein Profil); Anmeldung mit Code per Mail und Passkey; Einladung; Ersteinrichtung per Konsole mit Link und Passkey-Onboarding; App-Datenbank; Verbindungen mit Test, Verschlüsselung und erlaubten Hosts; Dialekt-Schicht mit Vertragstest auf vier Datenbanken; lokales Prüfskript, Workflow sparsam | A-01 bis A-06, A-09, V-01 bis V-03, V-05, B-01 bis B-03, B-06, B-08 | ✅ 28.10.2026 — offen: Touch ID von Hand (A-03), erster Lauf des Workflows auf GitHub |
| **01 Katalog und Navigator** | Arbeitsbildschirm mit Navigator für vier Datenbanken, «alle Datenbanken», Strukturansicht mit Art je Spalte, Schreibschutz und Umgebung, Kontowarnung, Deutsch und Englisch | O-01, O-04, S-01, V-04, N-06, V-10, O-06, N-04 | ✅ 29.10.2026 — V-06 mit TB 07; N-04-Matrix noch nie gelaufen |
| **02 Datenraster und Filter** | Raster mit Blättern über den Schlüssel, NULL zuletzt, Filtern, Suche, Zählen, Fremdschlüssel in beide Richtungen, Zeilen-Detail, Reiter, Kürzel; erste Seite bei 1 Mio. Zeilen unter 20 ms | D-01, D-02, D-03, D-06, N-01, O-03, O-05, O-09 | ✅ 29.10.2026 — «?»-Taste von Hand offen |
| 03 SQL-Editor und Befehlspalette | siehe Timebox-Plan | Q-01–Q-04, Q-09, O-02 | als Nächstes |

## Architekturentscheide

Ausformuliert in der Foundation am 28.10.2026 (Kontext → Optionen → Entscheid → Konsequenzen), Grundlage Feasibility und Solution-Architektur:

| ADR | Entscheid | Status |
|---|---|---|
| ADR-001 | Dialekt-Schicht: ein Vertrag, vier Umsetzungen | angenommen |
| ADR-002 | Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber | angenommen |
| ADR-003 | Vorschau aus `toSql()`, Anwenden je Treiber | angenommen |
| ADR-004 | Schreibschutz in Schichten, eine Anweisung je Aufruf | angenommen |
| ADR-005 | SQL-Editor CodeMirror 6 | angenommen |
| ADR-006 | Raster TanStack Virtual, Blättern über den Schlüssel | angenommen |
| ADR-007 | Export und Import in Paketen im Hintergrund | angenommen |
| ADR-008 | Anmeldung ohne Passwort | angenommen |
| ADR-009 | App-Datenbank SQLite als Vorgabe | angenommen |

| ADR | Entscheid | Status |
|---|---|---|
| ADR-010 | Lizenz MIT | angenommen |
| ADR-011 | PHP 8.5 als Mindestversion | angenommen (Hoang 28.10.2026) |

**Übernahme aus der Feasibility (erledigt in TB 00):** Die Proben F-1, F-7 und Teile von F-2 sind Vertragstests der Dialekt-Schicht (`tests/Feature/Dialects`); die Probe-Seiten `routes/probe.php`, `resources/js/pages/probe/*` bleiben lokal hinter der Anmeldung, bis Raster und SQL-Editor sie ersetzen — die Raster-Probe ist seit TB 02 entfernt, die Editor-Probe folgt mit TB 03.

# Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen

| Feld | Wert |
|---|---|
| Status | Angenommen |
| Datum | 28.10.2026 (Foundation; F-1, F-3) |
| Betrifft | Katalog, Struktur, SQL-Editor, Server-Verwaltung |

## Kontext

phpdbadmin soll vier Datenbanken mit derselben Bedienung abdecken (P-01). Laravel 13 liefert Verbindungen, Query Builder, Schema-Grammatiken und Introspektion für alle vier; die Feasibility zeigt, wo es nicht reicht: Check-Constraints, Trigger, Routinen, eigene Typen (`getTypes()` nur PostgreSQL), Prozesse, Rechte, Zeitlimits, Kontorechte. Ausserdem liefert MariaDB ohne Schemaangabe die Tabellen aller Datenbanken des Servers (3'603 statt 1'000).

## Optionen

| Option | Vorteile | Nachteile |
|---|---|---|
| A — eigene Abstraktion für alles | volle Kontrolle | baut nach, was Laravel schon kann und testet; viermal so viel Code |
| **B — Laravel zuerst, Dialekt nur für Unterschiede** | wenig eigener Code, Laravel-Updates kommen mit | Grenze zwischen Laravel und Dialekt muss klar sein |
| C — Paket eines Dritten (z. B. Adminer-Treiber) | viel fertig | fremde Struktur, Lizenzfrage, Bedienkonzept passt nicht |

## Entscheid

**Option B.** Ein Vertrag `App\Dialects\Dialect` mit vier Umsetzungen (`MySqlDialect`, `MariaDbDialect` als Ableitung, `PostgresDialect`, `SqliteDialect`). Was Laravel kann, ruft der Dienst direkt; der Dialekt liefert nur, was sich unterscheidet. Der Katalog übergibt **immer** das Schema.

## Konsequenzen

- Jede Dialekt-Methode hat einen Vertragstest, der gegen alle vier Herd-Dienste läuft; die Proben aus `tests/Feature/Feasibility` werden dazu umgebaut.
- Neue Datenbanken (SQL Server, R-14) bekommen eine fünfte Umsetzung, die Oberfläche bleibt.
- Typnamen erscheinen im Original und mit einem gemeinsamen Oberbegriff.

Siehe Solution-Architektur · Feasibility

# Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber

| Feld | Wert |
|---|---|
| Status | Angenommen |
| Datum | 28.10.2026 (Foundation; F-2, F-8) |
| Betrifft | ConnectionFactory, QueryRunner |

## Kontext

Zugangsdaten liegen verschlüsselt in der App-Datenbank. Jede Anfrage braucht eine Verbindung mit Zeitzone UTC, Zeitlimit und gegebenenfalls Schreibschutz. Die Feasibility zeigt grosse Unterschiede: MySQL begrenzt nur SELECT und bricht still ab (Ergebnis `1`), MariaDB und PostgreSQL melden 70100 bzw. 57014, SQLite kennt kein Zeitlimit; libpq hält ganze Ergebnisse ausserhalb des PHP-Speichers (+105 MB bei einer Million Zeilen).

## Optionen

| Option | Vorteile | Nachteile |
|---|---|---|
| A — Verbindungen in `config/database.php` eintragen | einfach | Neustart je Verbindung, Geheimnisse in Dateien |
| **B — `DB::build()` je Anfrage aus der App-Datenbank** | keine Dateien, Name je Verbindung und Konto, `DB::purge` danach | Verbindungsaufbau je Anfrage (lokal ~1 ms) |
| C — dauerhafte Verbindungen (Pool) | schneller | PHP-FPM hat keinen gemeinsamen Pool; Zustand zwischen Konten |

## Entscheid

**Option B.** `ConnectionFactory` baut je Anfrage mit `timezone`, Zeitlimit (`max_execution_time`, `max_statement_time`, `statement_timeout`) und Optionen. Zeilengrenze je Treiber: MySQL/MariaDB ungepuffert + `KILL QUERY`, PostgreSQL Server-Cursor (`DECLARE … CURSOR`, `FETCH`), SQLite streamt. SQLite-Abfragen aus dem Editor laufen im Arbeitsprozess mit Zeitgrenze. Abbrechen ist eine zweite Anfrage; ein Merker im Cache entscheidet über «abgebrochen», nicht der Fehlercode.

## Konsequenzen

- Verbindungsfehler werden je Treiber zugeordnet (MySQL nach Code, PostgreSQL nach Meldungstext).
- Der Arbeitsprozess für SQLite braucht die Warteschlange oder einen Kindprozess; ohne beides gilt die PHP-Laufzeitgrenze.
- Erlaubte Hosts werden vor dem Verbinden geprüft (ADR-004, V-05).

Siehe Solution-Architektur · Feasibility

# Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber

| Feld | Wert |
|---|---|
| Status | Angenommen |
| Datum | 28.10.2026 (Foundation; F-5) |
| Betrifft | SchemaEditor, ChangeSet |

## Kontext

Nichts soll ungesehen passieren (P-02). Die Feasibility belegt: `toSql()` ist auf allen vier genau das, was läuft; `pretend()` liefert für DML eine gute Vorschau, für SQLite-DDL aber eine kaputte. Naiv in einer Transaktion angewendet, wirft Laravel auf MySQL/MariaDB «There is no active transaction» (DDL beendet Transaktionen implizit), und **SQLite löscht beim Umbau einer Elterntabelle alle abhängigen Zeilen** (0 von 2), weil `PRAGMA foreign_keys` in einer Transaktion nicht wirkt.

## Optionen

| Option | Vorteile | Nachteile |
|---|---|---|
| A — alles in einer Transaktion | einfach, scheinbar sicher | auf drei von vier Treibern falsch, auf SQLite Datenverlust |
| **B — Verfahren je Treiber** | auf allen vier belegt korrekt (2 von 2 Kindern bleiben) | drei Wege, die Vorschau muss sie erklären |
| C — Strukturänderungen nur über eigenes SQL | keine Abhängigkeit von Laravel | baut die Grammatiken nach, die SQLite-Umbau schon können |

## Entscheid

**Option B.** Vorschau von Strukturänderungen aus `Blueprint::toSql()`, von Datenänderungen aus `pretend()`. Anwenden: MySQL/MariaDB Anweisung für Anweisung ohne Transaktion (Vorschau: «nicht rückrollbar»); PostgreSQL in einer Transaktion; SQLite nach sqlite.org: Fremdschlüssel ausserhalb aus, Umbau in der Transaktion, `foreign_key_check`, Commit, ein. Datenänderungen auf allen vier in einer Transaktion, vorher Sicherung für Rückgängig.

## Konsequenzen

- Tests für Strukturänderungen immer mit Elterntabelle und abhängigen Zeilen.
- Bei MySQL/MariaDB meldet die Oberfläche nach einem Fehler, welche Schritte schon gelaufen sind.
- Lektion «SQLite-Tabellenumbau nie in eigener Transaktion» im Vault.

Siehe Solution-Architektur · Feasibility

# Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf

| Feld | Wert |
|---|---|
| Status | Angenommen |
| Datum | 28.10.2026 (Foundation; F-7) |
| Betrifft | ReadOnlyGuard, QueryRunner, SQLite-Verbindungen |

## Kontext

Produktion ist standardmässig schreibgeschützt (V-04), und der SQL-Editor nimmt freies SQL. Ein Schreibschutz über eine Sitzungseinstellung allein lässt sich im Editor abschalten (`SET … READ WRITE`, `PRAGMA query_only = OFF`). Die Feasibility hat einen Wächter mit 5–12 Ausbruchsversuchen je Treiber geprüft: Bestand unverändert.

## Optionen

| Option | Vorteile | Nachteile |
|---|---|---|
| A — nur Lesekonto verlangen | das einzige echte Schloss | viele haben nur ein Konto; lokal unpraktisch |
| **B — Schichten: Lesekonto empfohlen, Nur-lesen-Transaktion je Anweisung, vorbereitet, SQLite nur lesend geöffnet + Authorizer** | belegt auf allen vier | jede Anweisung einzeln; Zerleger nötig |
| C — SQL vor dem Ausführen klassifizieren und Schreibendes ablehnen | ohne Transaktion | Parser-Rennen gegen jede Syntax; Funktionen mit Nebenwirkung |

## Entscheid

**Option B.** MySQL/MariaDB: `SET SESSION TRANSACTION READ ONLY` + Transaktion, immer zurückrollen; PostgreSQL: Transaktion + `SET TRANSACTION READ ONLY`, zurückrollen; SQLite: `Pdo\Sqlite::OPEN_READONLY` und `setAuthorizer()` (`ATTACH` nur im erlaubten Ordner). Anweisungen werden auf dem Server zerlegt und einzeln vorbereitet; `unprepared()` ist für Eingaben verboten. Konten mit Superuser- oder Dateirechten werden sichtbar gewarnt (V-10).

## Konsequenzen

- Der Zerleger je Dialekt ist Voraussetzung (TB 03, Probe zu Beginn).
- Die Ausbruchsversuche aus `SecurityProbeTest` werden Abnahmetest von V-04.
- PHP 8.5 wird Mindestversion (ADR-011).

Siehe Solution-Architektur · Feasibility

# Anhang E — ADR-005 — SQL-Editor: CodeMirror 6

| Feld | Wert |
|---|---|
| Status | Angenommen |
| Datum | 28.10.2026 (Foundation; F-4, Freigabe der Pakete durch Hoang) |
| Betrifft | Oberfläche SQL-Reiter |

## Kontext

Der Editor braucht Hervorhebung und Vervollständigung je Dialekt aus dem Schema (Q-01). Probeweise installiert: `codemirror` 6.0.2, `@codemirror/lang-sql` 6.10.0 (MIT, 17 Pakete). Belegt: Tabellen aus 1'002 Tabellen in höchstens 2 ms, Aliasse aufgelöst, dialekteigene Schlüsselwörter (`ILIKE` nur PostgreSQL); Chunk 425 KB (138 KB gzip).

## Optionen

| Option | Vorteile | Nachteile |
|---|---|---|
| **A — CodeMirror 6** | modular, MIT, Dialekte für alle vier, klein genug | eigene Vue-Hülle |
| B — Monaco | VS-Code-Gefühl | mehrere MB, SQL-Dialekte schwächer, Web-Worker |
| C — Eigenbau mit Textarea | keine Abhängigkeit | keine Vervollständigung, viel Arbeit |

## Entscheid

**Option A**, als eigener Chunk nur auf dem SQL-Reiter geladen.

## Konsequenzen

- Sichtprüfung im Browser durch Hoang steht aus (Probe-Seite `/probe/editor`).
- Bei Updates der Pakete Lizenzliste neu erzeugen (ADR-010).

Siehe Solution-Architektur · Feasibility

# Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel

| Feld | Wert |
|---|---|
| Status | Angenommen — bestätigt von Hoang am 28.10.2026 (Endlos scrollen statt Seiten zu 50) |
| Datum | 28.10.2026 (Foundation; F-3) |
| Betrifft | Datenraster |

## Kontext

Tabellen mit Millionen Zeilen müssen sofort erscheinen (N-01). Gemessen: erste Seite über den Schlüssel 0,1–0,6 ms in der Datenbank, `OFFSET 900000` 18–232 ms; im Browser 281 ms für die erste Seite, davon fast alles Laravel und Herd; im DOM bleiben rund 25 Zeilen.

## Optionen

| Option | Vorteile | Nachteile |
|---|---|---|
| **A — eigenes Raster mit** `@tanstack/vue-virtual` (MIT, 34 KB Chunk) | volle Kontrolle über Bearbeiten, Zustände, Gestaltung | Raster selbst bauen |
| B — fertiges Datenraster (AG Grid Community u. a.) | viel fertig | schwer, eigene Bedienlogik, Gestaltung schwer anzugleichen |
| C — ohne Virtualisierung, Seiten zu 100 | einfach | kein flüssiges Scrollen, Seitenwechsel |

## Entscheid

**Option A.** Seiten zu 200 Zeilen über den Schlüssel, die nächste vorgeladen; `OFFSET` nur ohne Primärschlüssel; erste Spalte fixiert, waagrecht scrollen nur hier.

## Konsequenzen

- Geänderte, neue und zu löschende Zeilen sind Zustand im Browser bis «Anwenden» (D-04).
- Kartenliste unter 768 px (G-07).

Siehe Solution-Architektur · Feasibility

# Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund

| Feld | Wert |
|---|---|
| Status | Angenommen |
| Datum | 28.10.2026 (Foundation; F-8, F-3) |
| Betrifft | Transfer |

## Kontext

Gemessen bei einer Million Zeilen: `lazyById(5000)` 3–3,3 MB auf allen vier; `cursor()` gepuffert 62 MB (MySQL/MariaDB), PostgreSQL +105 MB im Prozess. Import von 100'000 Zeilen in Paketen zu 1'000: 143–502 ms. Nach dem Füllen fehlt MySQL/MariaDB die Statistik — Sortieren über den Index 300 statt 2 ms.

## Optionen

| Option | Vorteile | Nachteile |
|---|---|---|
| **A — Pakete über den Schlüssel, im Hintergrund, Datei mit Frist** | gleich auf allen vier, wenig Speicher | Warteschlange nötig (ohne Worker synchron mit Grenze) |
| B — Server-Werkzeuge (`mysqldump`, `pg_dump`) | schnell, vollständig | nicht überall installiert, Versionen müssen passen — nur für Sicherung (E-03) |

## Entscheid

**Option A** für Export und Import; Probelauf beim Import in einer zurückgerollten Transaktion; nach eigenen Importen `ANALYZE` (E-06). Server-Werkzeuge nur für Sicherung und Wiederherstellung (E-03, v1.1).

## Konsequenzen

- Exportdateien unter `storage/app/private/transfers`, Frist 24 Stunden.
- Tabellen ohne Primärschlüssel exportieren über `cursor()` ungepuffert bzw. Server-Cursor.

Siehe Solution-Architektur · Feasibility

# Anhang H — ADR-008 — Anmeldung ohne Passwort

| Feld | Wert |
|---|---|
| Status | Angenommen |
| Datum | 28.10.2026 (Vorgabe Hoang; F-6) |
| Betrifft | Anmeldung, Konten |

## Kontext

Vorgabe: kein Passwort, nur Passkey und Code per Mail; kein Dashboard, kein Profil. Fortify 1.41 registriert `POST /login` und die Passwortbestätigung unabhängig von den Features; das Feature `passkeys` steht im Starter-Kit mit `confirmPassword: true`. Basalto hat dasselbe mit denselben Versionen gebaut (Code-Login, Abweisung, Software-Authenticator für Tests).

## Optionen

| Option | Vorteile | Nachteile |
|---|---|---|
| **A — Fortify nur für Passkeys, eigener Code-Login, Passwort-Endpunkte abweisen** | Muster erprobt, Passkeys von Laravel | Fortify-Updates können Routen verschieben (Lektion) |
| B — Fortify ganz entfernen, Passkeys selbst | keine Überraschungen aus Fortify | WebAuthn selbst prüfen — mehr Risiko |

## Entscheid

**Option A.** `Fortify::authenticateUsing()` liefert `null`; Passwortbestätigung durch Bestätigung per Passkey bzw. Code ersetzt; Passwort-Spalten und -Tabellen entfernt; erstes Konto per `phpdbadmin:install`, weitere per Einladung.

## Konsequenzen

- Test ruft jede alte Adresse des Starter-Kits auf (404/405).
- Vor jedem `composer update` Routenliste sichern und vergleichen.
- Parameter nie `{passkey}` nennen (globale Bindung von `laravel/passkeys`).

## Nachtrag TB 00 (28.10.2026)

Umgesetzt mit einer strengeren Form von Option A: `Fortify::ignoreRoutes()` statt `authenticateUsing()`. Fortify registriert damit **keine** eigenen Routen; `routes/auth.php` trägt die Passkey-Wege der Bibliothek (`Laravel\Passkeys\Http\Controllers`) unter den Namen, die Fortify verwendet hätte. Ein Passwort-Endpunkt existiert gar nicht (`POST /login` → 405). Die Middleware

`password.confirm` leitet auf eine eigene Seite (Bestätigen per Passkey oder Code an die eigene Adresse); jede Anmeldung setzt die Bestätigung. Fortify bleibt nur, weil es die Passkey-Konfiguration (`passkeys.*`) aus `config/fortify.php` ableitet.

**Ersteinrichtung (28.10.2026, Wunsch Hoang):** Das erste Konto entsteht im Browser oder per Konsole. Im Browser schützt ein Einrichtungsschlüssel, den die App beim ersten Aufruf von `/setup` als Datei auf dem Server ablegt (`storage/app/private/einrichtungsschluessel.txt`, 0600) und nach der Einrichtung löscht — so wird nicht der erste Besucher einer frisch installierten Seite Administrator (T-19). Bewusst nicht gewählt: offene Einrichtung für den ersten Besucher (Übernahme), Freigabe nach Absender-IP (hinter einem Reverse-Proxy kommt jede Anfrage von `127.0.0.1`), Bestätigung per Mail (auf einem frischen Server oft noch nicht eingerichtet). Nach der Einrichtung folgt das Onboarding mit Passkey.

Siehe Solution-Architektur · Feasibility

# Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe

| Feld | Wert |
|---|---|
| Status | Angenommen |
| Datum | 28.10.2026 (Foundation; R-31) |
| Betrifft | Installation, Betrieb |

## Kontext

Die App-Datenbank hält Konten, Verbindungen, Verlauf und Protokoll — wenig Daten, wenige gleichzeitige Schreiber. Eine Installation neben fremden Datenbanken soll ohne zusätzlichen Server laufen.

## Optionen

| Option | Vorteile | Nachteile |
|---|---|---|
| **A — SQLite als Vorgabe, die anderen drei möglich** | keine Einrichtung, Sicherung = eine Datei | Schreibsperren bei vielen Parallelzugriffen (`busy_timeout`, WAL) |
| B — MariaDB/PostgreSQL vorgeschrieben | robuster bei vielen Konten | zusätzlicher Server nur für phpdbadmin |

## Entscheid

**Option A**, mit WAL und `busy_timeout`; die Testsuite läuft mit jeder der vier als App-Datenbank (B-06).

## Konsequenzen

- Regeln für Spaltentypen gelten trotzdem für alle vier (Datenmodell § 1).
- Die App-Datenbank erscheint nie als Zielverbindung (R-17).

Siehe Solution-Architektur · Feasibility

# Anhang J — ADR-010 — Lizenz MIT

| Feld | Wert |
|---|---|
| Status | Angenommen |
| Datum | 28.10.2026 (Entscheid Hoang, R-40; F-9) |
| Betrifft | Repository, Abhängigkeiten |

## Kontext

Hoang hat MIT gewählt (Markt überwiegend kostenlos, Schaufenster für Nguyen Development). Geprüft: Composer 101 Pakete MIT/BSD/Apache, `nette/*` wahlweise BSD-3; npm-Laufzeit MIT/ISC/BSD/Apache; neue Pakete MIT.

## Optionen

| Option | Vorteile | Nachteile |
|---|---|---|
| **A — MIT** | einfach, verbreitet | kein Schutz gegen geschlossene Ableitungen |
| B — GPL-3 | Ableitungen bleiben offen | schreckt Einsatz in Firmen ab |

## Entscheid

**Option A.** `LICENSE` mit MIT vor dem ersten öffentlichen Push; Code aus GPL-Projekten (phpMyAdmin, phpPgAdmin, Adminer) wird nicht übernommen.

## Konsequenzen

- Lizenzliste vor jedem Release erneut erzeugen.
- Hinweis «nicht verbunden mit phpMyAdmin» in README und Website (F-10).

Siehe Solution-Architektur · Feasibility

# Anhang K — ADR-011 — PHP 8.5 als Mindestversion

| Feld | Wert |
|---|---|
| Status | Angenommen (Hoang, 28.10.2026) |
| Datum | 28.10.2026 (Foundation) |
| Betrifft | Installation, Hosting, `composer.json` |

## Kontext

Der Schutz von SQLite gegen `ATTACH` fremder Dateien braucht `Pdo\Sqlite::setAuthorizer()`. Geprüft in Herd: PHP 8.4.27 hat `Pdo\Sqlite`, aber keinen Authorizer; PHP 8.5.12 hat ihn. Laravel 13 verlangt nur PHP 8.3, `composer.json` steht auf `^8.3`.

## Optionen

| Option | Vorteile | Nachteile |
|---|---|---|
| **A — PHP 8.5 vorschreiben** | Schutz vollständig, ein Weg | Hosting mit PHP 8.4 fällt weg |
| B — PHP 8.4 erlauben, SQLite-`ATTACH` dort per Textprüfung sperren | breiter einsetzbar | Textprüfung ist schwächer; zwei Wege testen |
| C — PHP 8.4 erlauben, SQLite-Verbindungen dort nur lesend und ohne Editor | einfach | halbe Funktion |

## Entscheid

**Option A** (entschieden von Hoang am 28.10.2026). `composer.json` auf `^8.5` in TB 00; Herd läuft bereits mit 8.5.

## Konsequenzen

- Die Installationsanleitung nennt PHP 8.5 und `pdo_sqlite`, `pdo_mysql`, `pdo_pgsql`.

Siehe Solution-Architektur · Feasibility

# Anhang L — Timebox 00 — Fundament

**Stand:** 28.10.2026 · ✅ abgeschlossen — gestartet auf Hoangs «Ja, TB 00 starten», gebaut in fünf Scheiben nach dem Timebox-Plan § 2; während Scheibe 5 auf Hoangs Wunsch ergänzt um **Ersteinrichtung und Passkey-Onboarding**.

## Ziel

Ein sauberes Fundament ohne Starter-Kit-Reste: Anmeldung ohne Passwort, Verbindungen zu vier Datenbanken anlegen und testen, eine Dialekt-Schicht mit Vertragstests — geprüft gegen MySQL 8.4, MariaDB 10.11, PostgreSQL 18 und SQLite.

## Umfang

| Scheibe | Anforderungen | Ergebnis |
|---|---|---|
| 1 Starter-Kit bereinigen | A-01, A-04, A-05, B-01 | ✅ |
| 2 Anmeldung | A-02, A-03, A-06, A-09 | ✅ (A-03: Handtest mit Touch ID offen, Hoang) |
| 3 Verbindungen | V-01, V-02, V-03, V-05, B-06 | ✅ |
| 4 Dialekt-Schicht | B-02 (+ Grundlage V-04, V-10) | ✅ |
| 5 Qualität und Abnahme | B-03, B-08 | ✅ lokal; Workflow noch nie auf GitHub gelaufen (kein Remote) |
| Zusatz: Ersteinrichtung | A-06, A-03 | ✅ (Wunsch Hoang während Scheibe 5) |

Wireframe-Bereiche 1 (Anmeldung), 2 (Code), 3 (Verbindungen), 4 (Verbindung anlegen), 13 (Konto).

## Was entstanden ist

**Anmeldung ohne Passwort (ADR-008, Nachtrag).** Fortify registriert keine eigenen Routen mehr (`Fortify::ignoreRoutes()`); `routes/auth.php` trägt nur die Passkey-Wege der Bibliothek. `POST /login` antwortet 405 — einen Passwort-Endpunkt gibt es nicht. Code per Mail: sechs Ziffern, HMAC-Prüfsummen für Code, Adresse und IP, zehn Minuten, einmalig (bedingtes Update gegen zwei gleichzeitige Einlösungen), Sperre nach fünf Fehlversuchen, benannte Limiter je Adresse und IP, gleiche Antwort für unbekannte Adressen, die Mail geht erst nach der Antwort hinaus (`defer`), die Sitzung wird neu erzeugt. Der Code steht bewusst nicht im Betreff (Sperrbildschirm). Die Passwortbestätigung ist ersetzt durch «Bestätigen» per Passkey oder Code an die eigene Adresse; jede Anmeldung gilt als Bestätigung. Passkeys anlegen, umbenennen, entfernen im Konto. Protokoll: Anmeldungen (mit Weg: Code, Passkey, Einladung, Einrichtung), Fehlversuche (Adresse nur als Prüfsumme), Abmelden, Passkey-Änderungen.

**Ersteinrichtung und Onboarding.** Ohne jedes Konto zeigt `/login` die Ersteinrichtung statt einer Anmeldung ohne Ausweg. **Im Browser** (Hauptweg, Hoangs zweiter Wunsch: «nicht jeder hat ein Terminal»): `/setup` legt einen Einrichtungsschlüssel als Datei auf dem Server ab (`storage/app/private/einrichtungsschluessel.txt`, 0600, nie überschrieben, nach der Einrichtung gelöscht); wer die Installation hochgeladen hat, liest ihn über Dateimanager, FTP oder Editor und gibt ihn mit Name und E-Mail ein — ein zufälliger Besucher kann so nicht Administrator werden (T-19). **Per Konsole** (Alternative): `php artisan phpdbadmin:install` legt genau ein Administratorkonto an und gibt

einen signierten Link aus (30 Minuten, gilt nur bis zur ersten Anmeldung; erst ein Knopf meldet an, damit Link-Vorschauen ihn nicht verbrauchen). Die erste Anmeldung gelingt so auch, wenn der Server noch keine Mail versenden kann; solange die Ersteinrichtung offen ist, stellt der Befehl einen neuen Link aus. Wer sich ohne Passkey anmeldet (Code, Einladung, Einrichtungslink), landet im Onboarding «Passkey jetzt anlegen»; das erste Konto sieht die Variante «Ersteinrichtung» mit den nächsten Schritten. A-06 ist in CR-001 Fassung 5 entsprechend angepasst.

**Verbindungen.** Tabellen `connections` (ULID, Passwort mit `encrypted`-Cast) und `connection_user` nach dem Datenmodell. Anlegen, bearbeiten, entfernen für vier Treiber; Produktion schreibgeschützt vorgegeben; Passwort nie im Browser (die Seite erfährt nur, ob eines gespeichert ist), leeres Feld behält das gespeicherte, «nicht speichern» möglich. «Verbindung testen» mit Prüfschritten (Host erlaubt → Server erreichbar → Anmeldung → Rechte des Kontos), Fehler in Klartext je Treiber, Meldung des Servers zum Aufklappen. `HostPolicy`: Namen, Platzhalter, Adressen, Netze; einmal auflösen und zur geprüften Adresse verbinden; SQLite nur in erlaubten Ordnern über `realpath()`, die App-Datenbank nie. `{connection}` löst nur sichtbare Verbindungen auf — fremde sind 404 (T-16).

**Dialekt-Schicht (B-02).** Vertrag `App\Dialects\Dialect` mit Basis über Laravels Introspektion (Schema immer ausdrücklich) und vier Umsetzungen: Konfiguration (UTC, Zeitlimit beim Verbinden, SSL, Schreibschutz beim Öffnen bzw. für die Sitzung), Version, Fehlerzuordnung, Schemas, Tabellen und Sichten mit Zeilenschätzung, Check-Constraints und Trigger, Zeitlimit je Anweisung, Schreibschutz-Wächter (eine Anweisung je Aufruf in einer Nur-lesen-Transaktion, immer zurückgerollt), Kontorechte. `ConnectionFactory`: keine Mehrfachanweisungen, kein `LOCAL INFILE`, SQLite-`ATTACH` nur in erlaubten Ordnern (Authorizer).

**Gestaltung.** Palette aus `werkzeuge/kontraste.py` im Stylesheet (hell und dunkel), Inter und JetBrains Mono beim Build heruntergeladen und mit ausgeliefert (G-02), Wortmarke (G-06), Umgebungszeichen ○ ◐ ● mit Wort (G-03).

## Abnahmekriterien

| Anforderung | Kriterium (CR-001) | Nachweis |
|---|---|---|
| A-01 | alte Adressen 404/405, Spalten weg | `StarterKitRemovalTest`: 17 Adressen 404, `POST /login` 405, keine Spalten `password`, `two_factor_*`, keine Tabelle `password_reset_tokens` |
| A-02 | Erfolg, Ablauf, Wiederverwendung, falscher Code, Drosselung, unbekannte Adresse, Sitzung neu | `LoginCodesTest` (6), `LoginControllerTest` (14) — Sitzungs-ID vor/nach über das Cookie |
| A-03 | Zeremonie mit virtuellem Authenticator; einmal Touch ID | `PasskeyLoginControllerTest`, `PasskeyManagementTest`, `ConfirmControllerTest` mit dem Software-Authenticator (aus Basalto); **Touch ID von Hand: offen (Hoang)** |
| A-04 | `/dashboard` 404, Anmeldung führt zur Übersicht | `StarterKitRemovalTest`; Anmeldung mit Passkey → Übersicht, ohne Passkey → Onboarding |
| A-05 | `/settings/profile` 404, Kontomenü | Test + Browser-Durchlauf (Menü Sicherheit, Darstellung, Abmelden) |
| A-06 | Einladung annehmen, abgelaufene ablehnen; Befehl legt genau ein Admin-Konto an; falscher Schlüssel legt kein Konto an | `InvitationControllerTest` (7), `InstallCommandTest` (5), `InviteCommandTest` (3), `SetupControllerTest` (5), `FirstSetupControllerTest` (6), `SetupKeyTest` (4); im Browser mit falschem und richtigem Schlüssel durchgespielt |

| Anforderung | Kriterium (CR-001) | Nachweis |
|---|---|---|
| A-09 | Eintrag mit Zeit, Konto bzw. Adresse (gehasht), IP-Prüfsumme | Tests prüfen `auth.login`, `auth.failed` (ohne Klartext-Adresse), `auth.logout` |
| V-01 | je Treiber ein Test gegen den echten Dienst; Fehler am Feld | `ConnectionTesterTest` (Herd-Dienste), `ConnectionControllerTest` |
| V-02 | je Fehlerart ein Test mit erzwungenem Fehler | nicht erreichbar, Zugang verweigert (ohne Passwort in der Meldung), Datenbank fehlt — je Server; SSL (PostgreSQL), Host nicht erlaubt, Datei keine Datenbank |
| V-03 | kein Passwort in Props und JSON; Spalte verschlüsselt | Übersicht, Bearbeiten, Test-Endpunkt; Rohwert ≠ Klartext |
| V-05 | nicht erlaubter Host und Pfad (auch `..`, Verknüpfung) | `HostPolicyTest` (21 Fälle aus 5 Tests), Controller am Feld; im Browser `~/Development/../../../etc/hosts` abgewiesen |
| B-01 | Routen und Seiten ohne Starter-Kit-Reste | Routenliste: nur eigene und Passkey-Wege; Seitenordner `auth`, `connections`, `account`, `onboarding`, `probe` |
| B-02 | Vertragstest gegen alle vier Umsetzungen | `DialectContractTest`: 12 Tests, 32 Fälle über die Treiber — alle grün |
| B-03 | lokales Skript fährt alle vier; Workflow sparsam | `composer ci:local`; Workflow mit `paths-ignore`, `concurrency`, `timeout-minutes`, Matrix wöchentlich — **noch nie auf GitHub gelaufen** |
| B-06 | Suite läuft mit jeder App-Datenbank | `composer test:app-db`: grün mit SQLite, MariaDB, MySQL, PostgreSQL als App-Datenbank |
| B-08 | Prüfkette in der Reihenfolge der CI | `vp check` → `vue-tsc` → Pint → Larastan (Stufe 7) → PHPUnit |

**Stand der Suite:** 221 Tests, 209 bestanden, 12 übersprungen (die schweren Raster- und Export-Proben mit 1 Mio. Zeilen, bewusst nur auf Wunsch).

## Abweichungen von den Wireframes

| Bereich | Abweichung | Grund |
|---|---|---|
| 01 Anmeldung | Sprachwahl Deutsch · English fehlt | Englisch ist nicht Teil von TB 00 |
| 01 Anmeldung | neu: Ersteinrichtung, wenn es kein Konto gibt | Wunsch Hoang; noch nicht im Generator |
| 02 Code | «Code abgelaufen» hat keine eigene Ansicht; eine Meldung für falsch und abgelaufen | verrät nichts über den Zustand eines Codes |
| 02 Code | Hinweis ergänzt: «…sofern es dazu ein Konto gibt» | ehrlich bei gleicher Antwort für unbekannte Adressen |
| 03 Verbindungen | ohne «Zuletzt geöffnet», Favoriten, Befehlspalette, Klick in den Arbeitsbildschirm | kommen mit ihren Timeboxes (V-06, TB 01, TB 11) |
| 04 Verbindung anlegen | SSL ohne «Zertifikat prüfen» | verbunden wird zur geprüften Adresse (V-05); Prüfen des Zertifikats braucht eine eigene Anforderung |
| 13 Konto | ohne Sitzungen und Dichte | A-08 (TB 04), G-04 mit dem Raster (TB 01) |
| 13 Konto | Kontomenü ohne «Verwaltung» | TB 07 |
| neu | Ersteinrichtung (Link) und Onboarding «Passkey jetzt anlegen» | Wunsch Hoang; Wireframes dazu noch nicht im Generator |

## Browser-Durchlauf

Gegen einen gebauten Stand in einem Herd-Workspace (der eingebaute Browser erreicht den Vite-Dev-Server nicht): Bereiche 1–4 und 13 bei 1440 und 375 px, hell und dunkel, Anmeldung per Code mit echter Mail in Herd, Verbindung zu MariaDB anlegen und testen, abgewiesener Pfad mit `..`, Ersteinrichtung auf einer Installation ohne Konto. Gefunden und behoben: Adressen brachen mitten im Wort, Codefeld zu klein, Hinweis ohne Mail-Symbol und Ablaufzeit, Kopf «Verbindung testen» brach um, **Warnsymbol fehlte wegen eines vergessenen Imports** (jetzt meldet `vue-tsc` unbekannte Komponenten), «· Konto» ohne Datenbank, SQLite-Platzhalter mit einem hier nicht erlaubten Ordner, leeres Passwort galt als «gespeichert».

## Entscheide in der Timebox

| Entscheid | Begründung |
|---|---|
| `Fortify::ignoreRoutes()` statt `authenticateUsing()` | kein Passwort-Endpunkt statt eines abweisenden (ADR-008, Nachtrag) |
| Route `login.code.show` statt `login.code.form` | Wayfinder hängt verschachtelte Routen an die Funktion; `form` überschrieb die Formular-Variante und machte das Formular zum GET |
| kein «angemeldet bleiben» ( `remember_token` entfernt) | Datenmodell § 1; Sitzungsdauer genügt |
| Probe-Seiten bleiben lokal hinter der Anmeldung | Sichtprüfung des Editors durch Hoang offen; Raster (TB 01) und SQL-Editor (TB 05) ersetzen sie |
| Proben F-1, F-7 und Teile von F-2 → Vertragstests | Probe-Dateien entfernt; Abbrechen, DDL-Vorschau, Raster und Export bleiben Proben bis zu ihren Timeboxes |
| Lesekonto `phpdbadmin_lesen` und Datenbanken `phpdbadmin_test` , `phpdbadmin_app_test` auf den lokalen Herd-Servern | Vertragstest «warnt nicht» (V-10) und Suite mit App-Datenbank auf den Servern |

## Commits

`083e3c9` Scheiben 1+2 · `b04de27` Scheibe 3 · `ceab0e8` Scheibe 4 · `2bea65c` Scheibe 5 (Prüfskript, Workflow, Funde) · `95aa78c` Ersteinrichtung und Onboarding · `7336538` Protokoll · `bfb091c` symfony/mime · `fc75729` Ersteinrichtung im Browser.

## Offen

- **Touch ID von Hand** (A-03): einmal einen Passkey auf dem Mac anlegen und damit anmelden — Hoang.
- **Workflow auf GitHub:** noch kein Remote; der erste Push zeigt, ob beide Aufträge laufen.
- ~~**Sicherheitshinweis** `symfony/mime` (CVE-2026-45070, mittel)~~ — erledigt 28.10.2026 auf Hoangs Freigabe: 8.1.7 → 8.1.10, `composer audit` ohne Hinweise, Prüfkette auf allen vier App-Datenbanken grün, Testmail über Herd mit Umlauten im Betreff angekommen.
- Wireframes für Ersteinrichtung und Onboarding im Generator nachziehen — in Arbeit auf einem eigenen Zweig; dort fehlt noch die Einrichtung im Browser mit Einrichtungsschlüssel ( `fc75729` ).
- Sichtprüfung der Gestaltung durch Hoang am eigenen Bildschirm (G-01 bis G-08).

## Gelernt

- Textersetzungen per Skript immer mit Prüfung, ob die Stelle gefunden wurde — der Formatierer bricht Zeilen um, und eine stille Nicht-Ersetzung liess ein Symbol verschwinden (Lektion im Vault).
- `vue-tsc` meldet unbekannte Komponenten erst mit `checkUnknownComponents` (Lektion im Vault).
- Wayfinder: Routennamen nie auf `.form` enden lassen (Lektion im Vault).
- Der eingebaute Browser erreicht den Vite-Dev-Server nicht; für Sichtprüfungen einen Herd-Workspace mit Build nutzen und dessen `artisan dev` anhalten (Projekt-Memory).

# Anhang M — Timebox 01 — Katalog und Navigator

**Stand:** 29.10.2026 · ✅ abgeschlossen bis auf V-06 (Favoriten, «zuletzt geöffnet»), das mit TB 07 im selben Zweig gebaut wird, weil beides an `connection_user` hängt. Gestartet auf Hoangs «nächste tb»; ab dem Abend auf seinen Auftrag «mache alle TB durch» ohne Rückfragen weitergeführt.

## Ziel

Eine Verbindung öffnen und darin arbeiten können: Arbeitsbildschirm mit Navigator für alle vier Datenbanken, Struktur einer Tabelle in denselben Begriffen, Schreibschutz und Umgebung sichtbar, Konten mit zu vielen Rechten gewarnt — auf Deutsch und Englisch.

## Umfang

| Scheibe | Anforderungen | Ergebnis |
|---|---|---|
| 1 Arbeitsbildschirm öffnen | V-04, V-10 | ✅ Kopf mit Umgebung (Farbe, Zeichen, Wort), roter Streifen bei Produktion, Titel «PROD · …», Kontowarnung; Passwort je Sitzung, Fehlerkarte statt Fehlerseite |
| 2 Navigator | O-01 | ✅ Datenbank → Schema → Gruppen (Tabellen, Sichten, Routinen, Trigger, Ereignisse, Sequenzen) mit Filter «/»; 1'000 Tabellen in einem Test |
| Zusatz «Alle Datenbanken anzeigen» | V-01 | ✅ Wunsch Hoang; Fehler aus seinem Durchlauf (Endlosschleife) behoben |
| 3 Strukturansicht | S-01, O-04 | ✅ Wireframe 08 bei 1440 und 375 px |
| 4 Favoriten, zuletzt geöffnet | V-06 | ⏳ im Zweig von TB 07 |
| 5 Deutsch und Englisch | N-06, O-06 | ✅ alle Texte über das Sprachsystem, Test über alle Schlüssel |
| 6 Abnahme | N-04, O-04 | ✅ O-04 Bildschirmfotos; N-04 Untergrenzen in der Wochen-Matrix (noch nie gelaufen, kein Remote) |

Wireframe-Bereiche 3 (Verbindungen), 5 (Arbeitsbildschirm), 8 (Struktur).

## Was entstanden ist

**Arbeitsbildschirm** (`/c/{verbindung}`, `/c/{verbindung}/s/{schema}`). `TargetConnections` öffnet Zielverbindungen je Anfrage mit Zeitlimit (15 s) und Schreibschutz, wenn die Verbindung schreibgeschützt ist; jede lesende Anweisung läuft zusätzlich im Wächter des Dialekts. Fehler der Zielverbindung werden zur Fehlerkarte mit Ursache und nächstem Schritt, fehlende Passwörter («nicht speichern») fragt eine eigene Seite einmal je Sitzung ab. Fehler der eigenen App-Datenbank bleiben Fehler der App (früher als «Verbindung gescheitert» getarnt — im Durchlauf von TB 02 gefunden).

**Alle Datenbanken (Wunsch Hoang).** Ohne Datenbanknamen zeigt MySQL/MariaDB alle Datenbanken als Schemas, PostgreSQL öffnet `postgres` und bietet die übrigen Datenbanken im Navigator an (`?db=`). Der Fehler aus Hoangs Durchlauf («Maximum call stack size … Infinite recursion») lag in Laravels `DatabaseManager`: zwei Zielverbindungen mit gleichem Namen trennten sich gegenseitig, der Neuaufbau

traf das falsche Objekt. Zielverbindungen stehen seither in keinem globalen Register; der Neuverbinder erneuert genau dieses Objekt und richtet Schreibschutz, Authorizer und Zeitlimit wieder ein. Gegenprobe: Auf dem alten Stand hängen die neuen Tests in der Schleife.

**Struktur** (`…/o/{objekt}/structure`). `Structure` liest Spalten mit Typ im Original und einer **Art**, die in allen vier Datenbanken gleich heisst (`ColumnKind`: Zahl, Text, Auswahl, Wahrheitswert, Datum und Zeit, JSON, Binär, Kennung, Geometrie, Liste, Sonstiges), Indizes, Fremdschlüssel in beide Richtungen, Checks, Trigger und DDL. Gleichmacher für O-04: MariaDB-JSON (Alias für `longtext` mit `json_valid`) gilt als JSON, eigene Aufzählungstypen in PostgreSQL als Auswahl, Vorgaben als SQL-Literal (`'offen'` statt `offen` bzw. `'offen'::character varying`), MariaDBs Text «NULL» als keine Vorgabe.

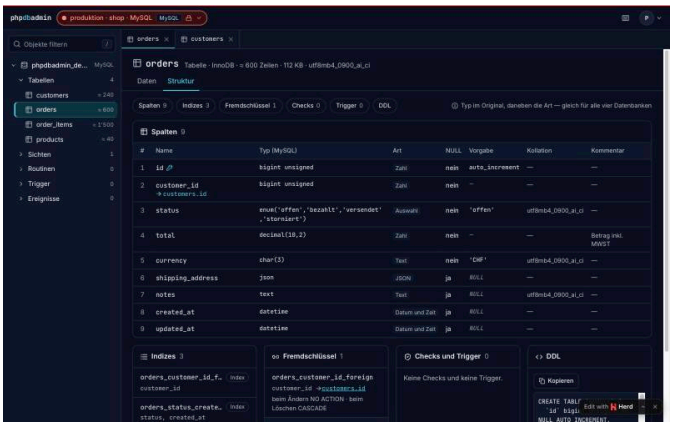
**Deutsch und Englisch (N-06, O-06).** Der deutsche Text ist der Schlüssel im Code (`__('…')`, `t('…')`, `tc('…')`), `lang/en.json` liefert Englisch (britisch). Sprache aus dem Konto, sonst aus dem Browser, sonst Deutsch; Mails in der Sprache des Kontos. Die Oberfläche bekommt die Übersetzungen einmal je Sprache (Once-Prop). Sprachwahl im Konto unter «Darstellung». Alle bestehenden Seiten hat ein Helfer in einer eigenen Arbeitskopie umgestellt (Zweig zusammengeführt, 322 Zeilen ohne Abweichung).

## Abnahmekriterien

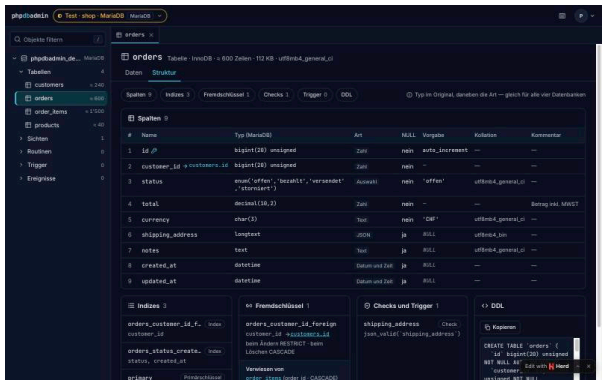
| Anforderung | Kriterium (CR-001) | Nachweis |
|---|---|---|
| V-04 | schreibgeschützte Verbindung lehnt jede schreibende Anweisung ab, auch Ausbruchsversuche | `DialectContractTest` (Wächter je Treiber), `TargetConnectionsTest` 19 Tests: Schreibschutz übersteht einen Neuaufbau (Laravel verbindet bei SQLSTATE 25006 neu) |
| V-10 | Herd-`root` warnt, Lesekonto nicht | `DialectContractTest` gegen root und `phpdbadmin_lesen` auf MySQL, MariaDB, PostgreSQL |
| O-01 | 1'000 Tabellen flüssig | `WorkspaceControllerTest::test_navigator_with_a_thousand_tables_stays_fast` (unter 1 s inkl. Seite) |
| O-04 | dieselbe Tabelle in allen vier Datenbanken | Bildschirmfotos unten; `StructureTest` 17 Tests (Art, Vorgaben, Fremdschlüssel beidseitig je Treiber) |
| S-01 | Beispieltabelle mit allen Merkmalen je Treiber | `StructureTest` (Kommentar, Aufzählung, JSON, Standardwerte, eindeutiger Index, Check, Trigger, Sicht), `ObjectControllerTest` |
| N-06 / O-06 | Sprachdateien vollständig ohne Rückfall | `LanguageFilesTest` (5): jeder Schlüssel mit englischer Zeile, keine verwaisten, gleiche Platzhalter, keine dynamischen Schlüssel, Feldnamen beider Sprachen; `LocaleControllerTest` (5) |
| N-04 | Untergrenzen | Wochen-Matrix «untergrenzen» (MariaDB 10.6, MySQL 8.0, PostgreSQL 14) — **noch nie gelaufen** (kein Remote); lokal nur aktuelle Versionen |

### O-04: dieselbe Tabelle `orders` in allen vier Datenbanken

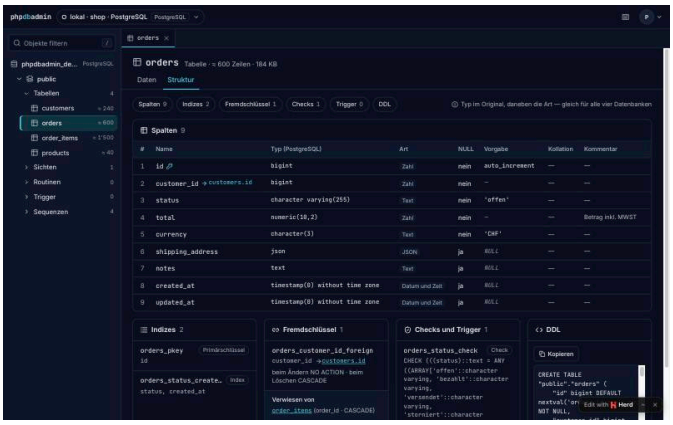
**MySQL 8.4 (Produktion, schreibgeschützt)**



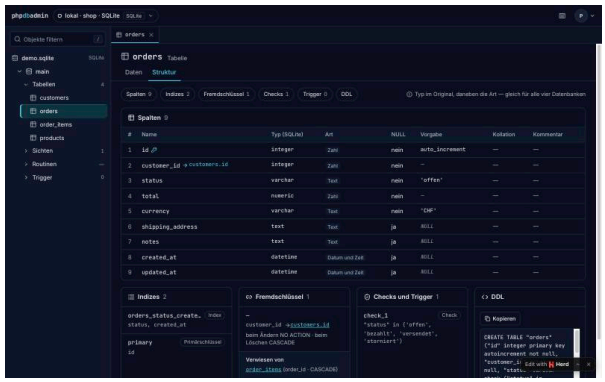
**MariaDB 10.11 (Test)**



**PostgreSQL 18 (lokal)**



**SQLite (lokal)**



Gleich: Aufbau, Begriffe, Art je Spalte, Vorgaben als Literal, Fremdschlüssel beidseitig. Erklärte Unterschiede: PostgreSQL legt Laravels `enum()` als `varchar` mit Check an (Art «Text», der Check steht daneben), SQLite kennt weder Zeilenschätzung noch Kollation noch Namen für Fremdschlüssel.

## Abweichungen von den Wireframes

| Bereich | Abweichung | Grund |
|---|---|---|
| 08 Kopf | Grösse als eine Zahl statt «Daten · Indizes» | Laravel liefert die Gesamtgrösse; getrennte Werte bräuchten je Treiber eigene Abfragen — später, falls gewünscht |
| 08 Knöpfe | «Tabelle ändern», «Exportieren», «Im Editor öffnen» fehlen | kommen mit TB 05, TB 06, TB 03 |
| 08 Reiter | «Beziehungen» und «SQL» fehlen | Beziehungsdiagramm S-07 (TB 10), SQL-Editor (TB 03) |
| 05 Kopf | Suchfeld ⌘K fehlt | Befehlspalette O-02 (TB 03) |

## Tests

Neu oder erweitert: `StructureTest` (17), `ColumnKindTest` (25), `ObjectControllerTest` (Struktur-Teil), `TargetConnectionsTest` (+3, unabhängige Verbindungen, Schlüssel je Datenbank), `WorkspaceControllerTest` (+1, Ablauf aus Hoangs Durchlauf), `DialectContractTest` (+1, VACUUM trotz

ATTACH-Wächter), `LanguageFilesTest` (5), `LocaleControllerTest` (5) und die Englisch-Tests des Übersetzungs-Helfers. Zielserver-Tests laufen über `PHPDBADMIN_TEST_DATABASE` auch parallel in mehreren Arbeitskopien.

## Commits

`d4766fa` Scheiben 1+2 · `0842694` Dialekt · `777ce66` alle Datenbanken · `a35ad37` Endlosschleife · `3dd2e9b` Struktur lesen · `884bf8e` Testdatenbank einstellbar · `dcf9e21` Sprachgrundlage · `6db5d35` Strukturansicht · `6a72d92` · `d2227f5` VACUUM · `558b4ae` Vorgaben, Chip · `c503ced` – `7f4c944` Sprache (Helfer) · `80fed1d` Merge · `8069824` Untergrenzen · `1c8f969` MariaDB-Vorgabe.

## Gelernt

- Laravels `DB::build()` mit gleichem Namen ersetzt die vorhandene Verbindung; `DB::reconnect()` erneuert das Objekt im Register, nicht das in der Hand. Zielverbindungen deshalb ausserhalb des Registers bauen (Lektion im Vault).
- «Connection refused» hält Laravel für eine verlorene Verbindung und versucht neu — ohne Neuverbinder ein `LostConnectionException`. Sofort verbinden, dann kommt der Fehler als PDO-Fehler.
- Ein leeres PHP-Array wird zu JSON `[]`, und `[].sort` ist in JavaScript eine Methode: Zuordnungen an die Oberfläche als Objekt senden.
- Symfony schickt in Tests `Accept-Language: en-us` mit — mit Sprachwahl aus dem Browser kippen sonst alle deutschen Erwartungen.

# Anhang N — Timebox 02 — Datenraster und Filter

**Stand:** 29.10.2026 · ✅ abgeschlossen — auf Hoangs Auftrag «mache alle TB durch» ohne Rückfragen gebaut.

## Ziel

Die Daten jeder Tabelle und Sicht schnell und in allen vier Datenbanken gleich sehen: blättern, sortieren, filtern, suchen, zählen, Fremdschlüsseln folgen — auch bei einer Million Zeilen.

## Umfang

| Scheibe | Anforderungen | Ergebnis |
|---|---|---|
| 1 Datendienst | D-01, D-02, D-03, D-06 | ✅ `TableData`, `RowDetail`, `Cells` — 52 Tests über vier Treiber |
| 2 Raster und Zeilen-Detail | D-01–D-03, D-06, O-09 | ✅ Wireframes 05 und 07 (Detail lesend; Bearbeiten mit TB 04) |
| 3 Reiter und Kürzel | O-03, O-05 | ✅ |
| Messung | N-01 | ✅ unter 20 ms für die erste Seite |

## Was entstanden ist

**Datendienst.** Blättern über den Primärschlüssel (zusammengesetzt ausgeschrieben, weil nicht jede Version Zeilenwerte vergleicht), sonst über den Versatz; der Primärschlüssel ist immer letztes Sortierkriterium. **NULL steht in beiden Richtungen zuletzt** — ausdrücklich über `case when … is null`, nicht nach Gewohnheit des Treibers. Geführte Filter (=, ≠, <, ≤, >, ≥, enthält, enthält nicht, beginnt, endet, ist NULL, ist nicht NULL) mit und/oder; alle Werte gebunden, `%` und `_` im Wert als Zeichen (Escape `!`). «enthält» ohne Gross-/Kleinschreibung: PostgreSQL `ILIKE` auf den Text der Spalte, MySQL/MariaDB nach Kollation (meist auch ohne Akzente), SQLite über eine Unicode-Faltfunktion (`phpdbadmin_fold`), die die `ConnectionFactory` registriert — sonst fände «MÜLLER» nicht «Müller». Freies WHERE als Klammerausdruck im Schreibschutz-Wächter mit Zeitlimit; Suche über alle Textspalten. Zählen genau bis `PHPDBADMIN_EXACT_COUNT_LIMIT` (50'000), darüber die Schätzung des Servers, gefiltert auf grossen Tabellen offen mit «genau zählen». Werte gleich in allen vier: Wahrheitswerte `true`/`false` (MySQL liefert 1/0), Binärdaten als Grösse und Anfang, lange Texte gekürzt, Ganzzahlen über 2^53 als Text, PostgreSQL-`bytea` aus dem Strom gelesen. Fremdschlüssel bekommen die Beschriftung der Zielzeile (bevorzugt `name`, `title`, `email` …). `TrustedSql` kennzeichnet die zwei Stellen mit absichtlich rohem SQL.

**Raster** (`/c/{verbindung}/s/{schema}/o/{objekt}`). Erste Seite kommt mit der Seite, weitere beim Scrollen (`rows`), dazu `count`, `row` und `view`. Virtualisiert (TanStack Virtual), Sortieren per Klick (Umschalt fügt hinzu), Breite ziehen, Spalten ausblenden — beides je Konto gemerkt (`table_views`). Tastatur:

Pfeile, Pos1/Ende, Bild auf/ab, Eingabe öffnet die Zeile. Filterleiste mit Chips, freiem WHERE, Suche und Zeilenzahl. Zeilen-Detail als Seitenblatt mit «verweist auf … öffnen» und «wird verwiesen von … 3 Zeilen zeigen». Telefon: Karten mit «alle Felder».

**Zustände (O-09).** Leere Tabelle, kein Treffer mit «Filter zurücksetzen», abgelehntes WHERE mit Meldung des Servers, Zeitlimit, abgerissene Verbindung und abgelaufene Sitzung beim Nachladen — je mit Ursache und nächstem Schritt.

**Reiter (O-03).** Jede geöffnete Tabelle oder Sicht wird ein Reiter mit ihrer letzten Adresse samt Filtern; gespeichert im Browser je Verbindung, übersteht das Neuladen. **Kürzel (O-05):** Übersicht über «?» oder das Tastatur-Symbol.

## Abnahmekriterien

| Anforderung | Kriterium (CR-001) | Nachweis |
|---|---|---|
| D-01 | Tests je Treiber mit Sortierung inkl. NULL, Reihenfolge ausdrücklich | `TableDataTest`: NULL zuletzt in beiden Richtungen, absteigend = Umkehrung von aufsteigend, Blättern ohne Lücken und Doppel (einfacher und zusammengesetzter Schlüssel, ohne Schlüssel), stabiler Versatz |
| D-02 | je Treiber, mit Umlauten und Werten, die wie SQL aussehen | `x' or '1'='1`, `'; drop table people; --`, `%`, `_`, «müller»/«MÜLLER»/«jürg», und/oder, NULL, freies WHERE inkl. `1 = 1; delete from people` (nichts gelöscht) |
| D-03 | Test je Treiber | Beschriftung des Fremdschlüssels, verweisende Zeilen gezählt, Zeile mit Verweisen in beide Richtungen |
| D-06 | Test mit Schwelle | geschätzt über der Grenze, genau darunter, gefiltert offen, «genau zählen»; SQLite zählt mangels Schätzung immer genau |
| N-01 | Messung mit erzeugten Daten je Treiber | `GridPerformanceTest` (`PHPDBADMIN_LEISTUNG=1`), Zahlen unten |
| O-03 | Durchlauf | im Browser: zwei Reiter, Neuladen, beide da |
| O-05 | Durchlauf nur mit Tastatur | Raster mit Pfeilen, Eingabe, Escape; Kürzel-Dialog über das Symbol. Die Taste «?» selbst liess sich mit dem Browser-Werkzeug nicht auslösen (es erzeugt kein `keydown`) — **von Hand offen** |
| O-09 | leere Tabelle, SQL-Fehler, getrennte Verbindung, abgelaufene Sitzung | `ObjectControllerTest` (ungültiges WHERE → Fehlerkarte, 401/404/422/503 der JSON-Antworten), Oberfläche erklärt jede Ursache |

### N-01: erste Seite einer Tabelle mit einer Million Zeilen (lokal, Herd)

| Fall | MySQL 8.4 | MariaDB 10.11 | PostgreSQL 18 | SQLite |
|---|---|---|---|---|
| ohne Sortierung | 6,5 ms | 6 ms | 17,1 ms | 6,6 ms |
| `created_at` absteigend (Index) | 1,5 ms | 2,3 ms | 2,4 ms | 5,9 ms |
| Filter `category = 42` (ohne Index) | 4,5 ms | 4,5 ms | 20,3 ms | 35,5 ms |
| Suche über Textspalten | 188,5 ms | 138 ms | 75,4 ms | 156,7 ms |

Zeit im Dienst inklusive Spalten, Zählung bzw. Schätzung und Beschriftungen.

## Abweichungen von den Wireframes

| Bereich | Abweichung | Grund |
|---|---|---|
| 05 Kopf | «Zeile einfügen», «Exportieren», Änderungsleiste, «Rückgängig machen» fehlen | Bearbeiten TB 04, Export TB 06 |
| 05 Navigator | «Gespeichert» und «Verlauf» fehlen | SQL-Editor TB 03 |
| 07 Detail | lesend; «Duplizieren», «Zum Löschen markieren», «Zeile kopieren als» fehlen | TB 04 (D-04, E-05) |
| 05 Raster | Spaltenkopf mit Typ statt Kurzform «decimal(10,2)» gekürzt | Breite der Spalte; der volle Typ steht in der Struktur |

## Tests

`TableDataTest` (52 = 13 × 4 Treiber), `ObjectControllerTest` (13), `GridPerformanceTest` (4, nur auf Wunsch). Suite: 393 Tests, 381 bestanden, 12 übersprungen (schwere Proben).

## Commits

`88a95d7` Datendienst · `b2842e7` Raster und Detail · `88dec69` Reiter und Kürzel · `8af0231` Messung N-01. Die Raster-Probe der Feasibility (`/probe/raster`) ist entfernt.

## Gelernt

- `rsync -R --delete` löscht auch in den Elternordnern des relativen Pfads — im Prüf-Workspace verschwanden so `database/database.sqlite`, Factories und Seeder (nur dort; Lektion im Vault).
- Larastan verlangt für rohes SQL ein Literal; statt die Prüfung zu umgehen, markiert `TrustedSql` die bewussten Stellen sichtbar.
- PostgreSQL schneidet als Text gebundene Binärdaten am Nullbyte ab — Testdaten für `bytea` als Literal setzen.